



*Université Polytechnique de
Bingerville*

**MASTER 1
DATA-IA
MODULE : BIG DATA**

RAPPORT D'INSTALLATION ET DE CONFIGURATION DE HADOOP SUR AWS EC2 EN MODE PSEUDO-DISTRIBUÉ

Réalisé par :

YAO MIÉZAN SAM WILLIAM

Enseignant :

Dr BOBET GOUALO

SOMMAIRE

1. Introduction	1
1.1 Configuration utilisée	1
1.2 Tableau de synthèse du processus	1
2. Présentation des technologies utilisées	2
2.1 Présentation d'Amazon Web Services (AWS)	2
2.2 Présentation d'Apache Hadoop	2
2.3 L'écosystème Hadoop et ses composants	3
3. Création de l'instance EC2	5
3.1 Choix de l'AMI et du type d'instance	5
3.2 Création de la paire de clés SSH	5
3.3 Configuration du groupe de sécurité	6
3.4 Configuration du stockage	7
3.5 Lancement et démarrage de l'instance	8
4. Connexion SSH à l'instance	9
4.1 Configuration de PuTTY	9
4.2 Connexion établie	9
5. Installation de Java 11	10
6. Installation de Hadoop et des variables d'environnement	11
6.1 Téléchargement et extraction de Hadoop 3.4.3	11
6.2 Configuration des variables d'environnement	11
7. Configuration des Fichiers Hadoop	12
7.1 Fichiers de configuration créés	12
8. Configuration SSH local et formatage NameNode	14
8.1 Configuration SSH localhost	14
8.2 Formatage du NameNode	14
9. Premier démarrage et détection du problème OOM	16
9.1 Résultat du premier démarrage	16
9.2 Diagnostic : OOM Killer Linux	16
10. Correctif Mémoire	18
10.1 Arrêt des daemons et ajout de swap	18
10.2 Réduction des heaps Java	18
10.3 Limitation mémoire des conteneurs YARN	18
11. Démarrage final et validation du Cluster	20
11.1 Vérification des services Hadoop via les interfaces Web	20
11.2 Deuxième démarrage avec 5 daemons actifs	21
11.3 Test pratique HDFS (écriture et lecture)	22
12. Conclusion	23

1. Introduction

Ce rapport présente le processus d'installation et de configuration d'Apache Hadoop en mode pseudo-distribué sur une instance Amazon EC2.

L'objectif de ce travail pratique est de déployer un cluster Hadoop single-node fonctionnel sur le cloud AWS, capable de gérer un système de fichiers distribué (HDFS) et d'orchestrer des traitements via YARN (Yet Another Resource Negotiator). L'ensemble des étapes de la création de l'instance EC2 jusqu'au test de lecture/écriture HDFS (Hadoop Distributed File System) est illustré par des captures d'écran inclus dans le document.

1.1 Configuration utilisée

Composant	Valeur
Fournisseur Cloud	Amazon Web Services (AWS)
Service	Amazon EC2 (Elastic Compute Cloud)
Région	af-south-1 (Afrique du Sud — Le Cap)
Type d'instance	t3.micro (2 vCPU, 1 Go RAM)
Système d'exploitation	Ubuntu Server 26.04 LTS (HVM), 64 bits
Stockage	20 Go SSD (gp3)
Version Hadoop	3.4.3 (mode pseudo-distribué)
Version Java	OpenJDK 11.0.31
Client SSH	PuTTY (Windows)

1.2 Tableau de synthèse du processus d'installation

Partie	Étape
Partie 1	Création instance EC2 (Ubuntu 26.04, t3.micro, af-south-1)
Partie 2	Connexion SSH via PuTTY (Windows)
Partie 3	Installation Java 11 (OpenJDK 11.0.31)
Parties 4-5	Téléchargement Hadoop 3.4.3 + variables d'environnement
Partie 6	Configuration des 4 fichiers XML Hadoop
Partie 7	Configuration SSH localhost
Partie 8	Formatage NameNode HDFS
Partie 9a	Premier démarrage avec OOM killer détecté
Partie 9b	Correctif mémoire (swap 2 Go + réduction heap)
Partie 9c	Deuxième démarrage avec 5 daemons actifs
Partie 10	Test HDFS écriture/lecture ("Hello Hadoop")

2. Présentation des technologies utilisées

Avant de détailler les étapes techniques de l'installation, il est utile de présenter brièvement les deux technologies au cœur du travail réalisé: Amazon Web Services, le fournisseur cloud qui héberge l'infrastructure, et Apache Hadoop, le framework Big Data qui y est déployé, ainsi que l'écosystème logiciel plus large dans lequel il s'inscrit.

2.1 Présentation d'Amazon Web Services (AWS)



Figure 1 : Logo d'Amazon Web Services (AWS)

Amazon Web Services (AWS) est la plateforme de cloud computing du groupe Amazon, lancée en 2006. C'est aujourd'hui le fournisseur cloud le plus utilisé au monde, avec plus de 200 services proposés à la demande : calcul, stockage, bases de données, réseau, intelligence artificielle, et bien d'autres. Le modèle de facturation repose sur le paiement à l'usage (pay-as-you-go), sans investissement matériel initial.

Dans le cadre de ce TP, AWS est utilisé comme infrastructure d'hébergement pour faire tourner Hadoop sur une machine virtuelle distante plutôt que sur un poste local. Trois services AWS interviennent dans ce déploiement :

Service AWS	Rôle dans ce TP
Amazon EC2 (Elastic Compute Cloud)	Fournit la machine virtuelle (instance) sur laquelle Hadoop est installé et exécuté
Groupe de sécurité (VPC)	Pare-feu réseau contrôlant l'accès à l'instance (ports 22, 9870 et 8088)
Amazon EBS (stockage)	Fournit le volume de stockage disque de l'instance (20 Go SSD gp3)

2.2 Présentation d'Apache Hadoop

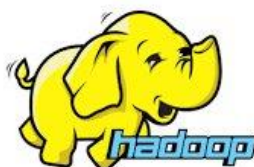


Figure 2 : Logo d'Apache Hadoop

Apache Hadoop est un framework open-source de la fondation Apache, conçu pour le stockage et le traitement distribué de très grands volumes de données sur des clusters de machines standard. Il a été créé initialement par Doug Cutting et Mike Cafarella en 2006, en s'inspirant des publications de Google sur le système de fichiers distribué GFS et le modèle de programmation MapReduce.

Hadoop repose sur deux composants fondamentaux : HDFS (Hadoop Distributed File System), qui découpe les fichiers en blocs et les réplique sur plusieurs noeuds afin de garantir disponibilité et tolérance aux pannes, et YARN (Yet Another Resource Negotiator), qui gère l'allocation des ressources de calcul (CPU, mémoire) et orchestre l'exécution des traitements sur le cluster. Ce sont précisément ces deux briques qui ont été installées et configurées dans ce TP, en mode pseudo-distribué sur une seule instance EC2.

2.3 L'écosystème Hadoop et ses composants

Au-delà de HDFS et YARN, Hadoop s'inscrit dans un écosystème logiciel beaucoup plus large, composé de nombreux projets complémentaires, chacun spécialisé dans un type de traitement ou d'usage particulier. Le schéma ci-dessous présente les principaux composants de cet écosystème, organisés autour des deux couches fondamentales que sont le stockage (HDFS) et la gestion des ressources (YARN).

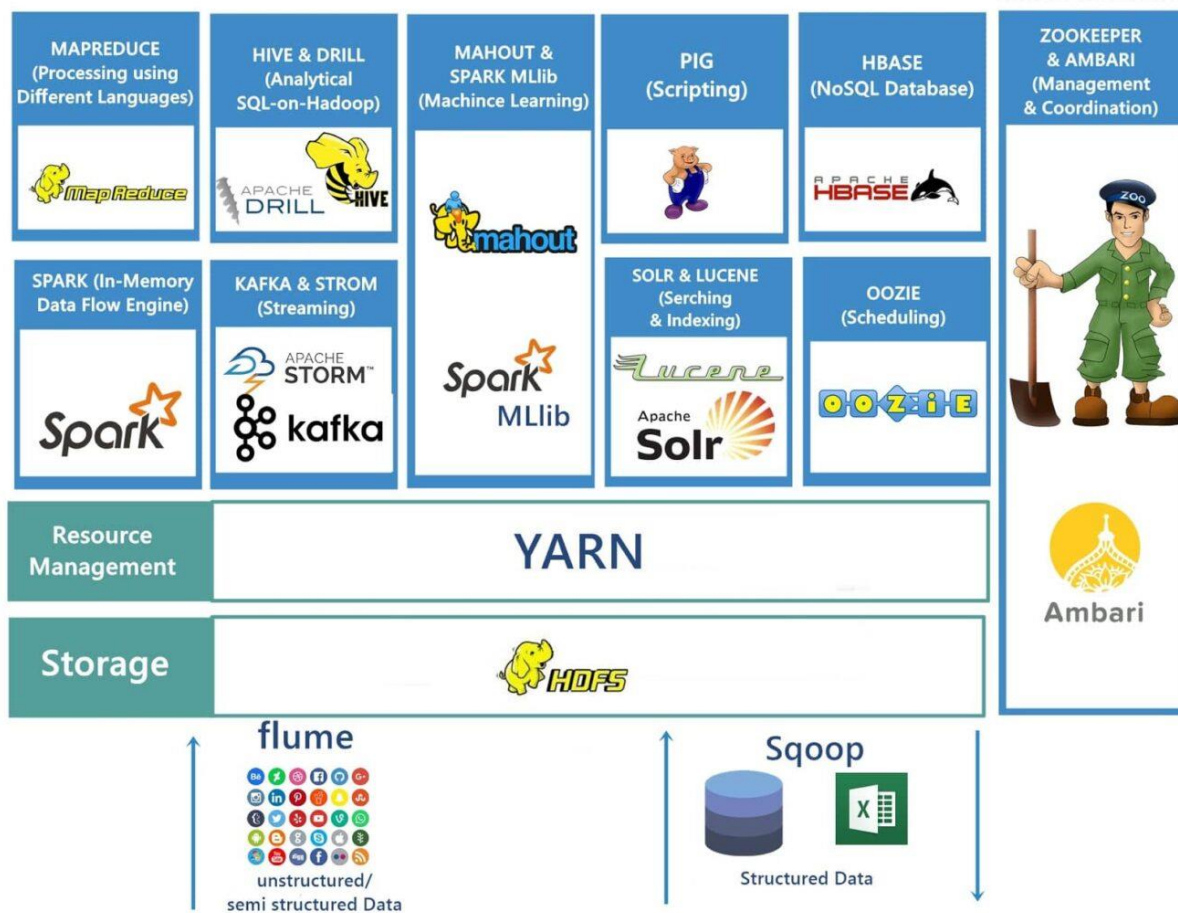


Figure 3 : Écosystème Hadoop montrant les composants organisés autour des couches HDFS (stockage) et YARN (gestion des ressources)

Composant	Catégorie	Rôle
MapReduce	Traitement par lots	Paradigme de calcul distribué historique de Hadoop, basé sur les phases Map et Reduce
Spark	Traitement en mémoire	Moteur de traitement rapide effectuant les calculs en mémoire plutôt que sur disque
Hive & Drill	SQL-on-Hadoop	Interrogation des données stockées dans HDFS via un langage proche du SQL
Pig	Scripting	Langage de script de haut niveau pour transformer des flux de données
Mahout & Spark MLlib	Machine Learning	Bibliothèques d'algorithmes d'apprentissage automatique distribué
HBase	Base NoSQL	Base de données orientée colonnes, accès en lecture/écriture en temps réel
Kafka & Storm	Streaming	Traitement de flux de données en temps réel
Solr & Lucene	Recherche et indexation	Moteur de recherche full-text et d'indexation de documents
Oozie	Orchestration	Planification et enchaînement de workflows de traitement Hadoop
Zookeeper & Ambari	Coordination et administration	Coordination des services distribués et supervision du cluster
Flume	Ingestion	Collecte de données non structurées ou semi-structurées (réseaux sociaux, journaux)
Sqoop	Ingestion	Import/export de données structurées entre Hadoop et des bases relationnelles

Dans le cadre de ce TP, seules les deux couches fondamentales HDFS pour le stockage et YARN pour la gestion des ressources ont été installées et configurées manuellement, comme détaillé dans les sections suivantes. Les autres composants présentés ci-dessus s'appuient sur cette même base et pourraient être ajoutés ultérieurement selon les besoins du traitement à réaliser.

3. Création de l'instance EC2

La première étape consiste à créer et configurer une machine virtuelle (instance) sur Amazon EC2. Cette instance constituera le serveur sur lequel Hadoop sera installé et exécuté.

3.1 Choix de l'AMI et du type d'instance

Depuis la console AWS (région af-south-1), on accède à la section EC2 > Instances > Lancer une instance. On sélectionne l'image Ubuntu Server 26.04 LTS et le type d'instance t3.micro, éligible au Free Tier AWS.

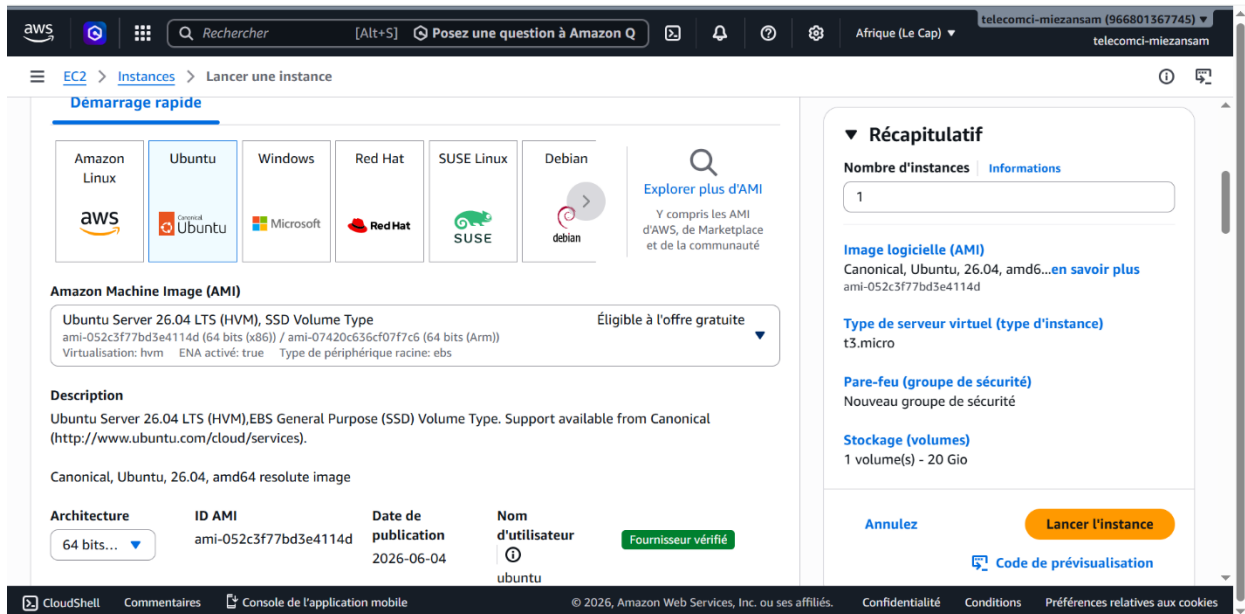


Figure 4 : Console AWS (page de lancement d'instance EC2, sélection Ubuntu 26.04 + t3.micro)

Remarque : Le type t3.micro dispose de seulement 1 Go de RAM. Cette contrainte nécessitera un correctif mémoire (ajout de swap) lors du démarrage des daemons Hadoop, documenté dans la partie 9b du rapport (Correctif Mémoire).

3.2 Création de la paire de clés SSH

Une paire de clés RSA nommée hadoop-key est créée pour sécuriser la connexion SSH à l'instance. Le format .ppk est sélectionné car l'outil PuTTY est utilisé sur Windows. Ce fichier doit être téléchargé immédiatement car après il est impossible de le récupérer.

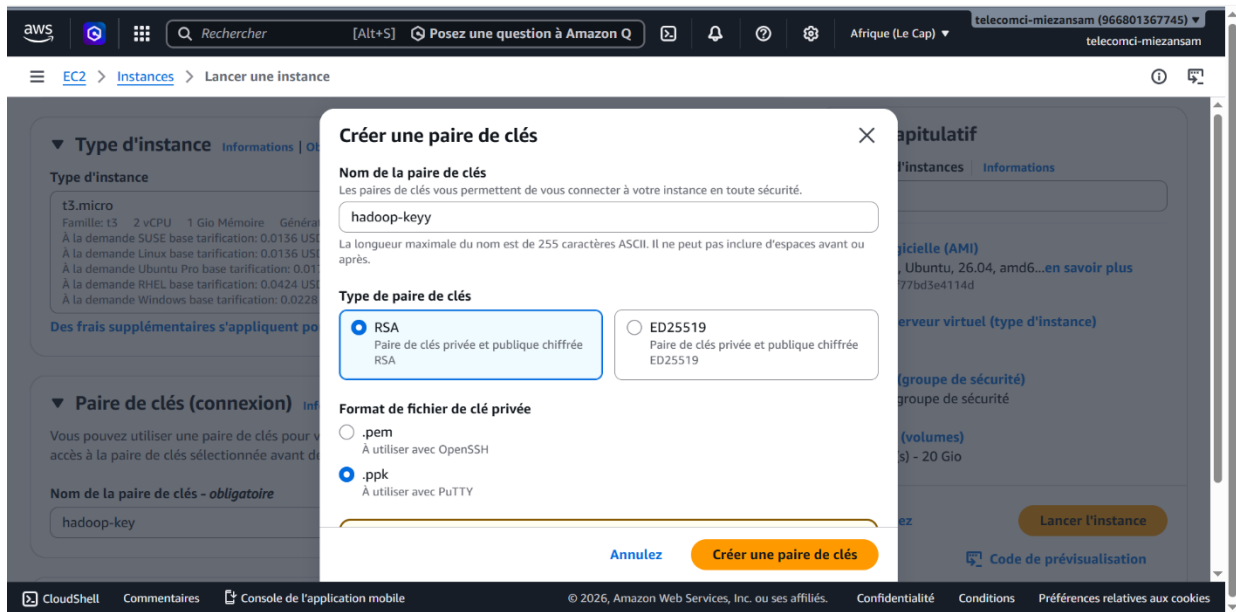


Figure 5 : Paramètres de création de la paire de clés

3.3 Configuration du groupe de sécurité (pare-feu)

Trois règles de trafic entrant sont configurées dans le groupe de sécurité, permettant l'accès SSH et aux interfaces web de Hadoop :

- Port 22 (SSH) : connexion sécurisée à l'instance depuis le poste local
- Port 9870 (TCP) : interface web du NameNode HDFS
- Port 8088 (TCP) : interface web du ResourceManager YARN

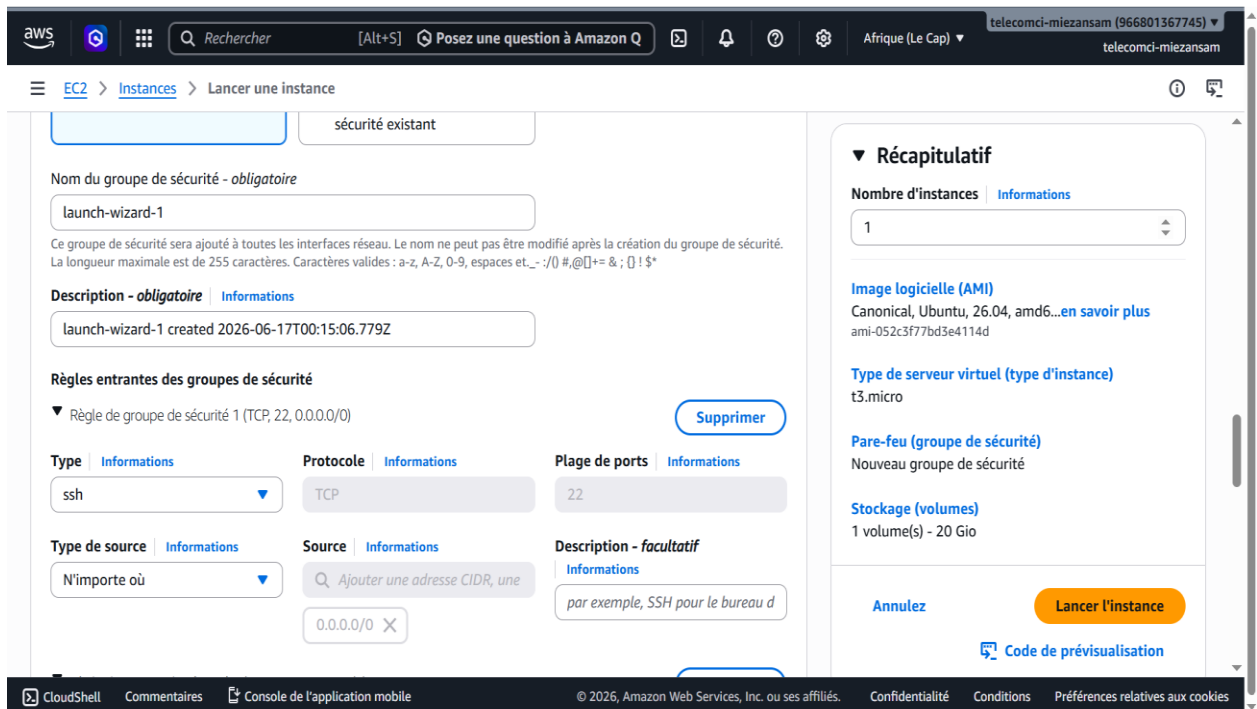


Figure 6 a : Groupe de sécurité 1 (règle SSH port 22)

Rapport d'Installation Hadoop sur AWS EC2

The screenshot displays the AWS Management Console interface for configuring an EC2 instance. The main section shows two security group rules being configured:

- Règle de groupe de sécurité 2 (TCP, 9870, 0.0.0.0/0):** Type: TCP personnalisé, Protocole: TCP, Plage de ports: 9870, Type de source: N'importe où, Source: 0.0.0.0/0.
- Règle de groupe de sécurité 3 (TCP, 8088, 0.0.0.0/0):** Type: TCP personnalisé, Protocole: TCP, Plage de ports: 8088, Type de source: N'importe où, Source: 0.0.0.0/0.

The summary (Récapitulatif) on the right indicates:

- Nombre d'instances: 1
- Image logicielle (AMI): Canonical, Ubuntu, 26.04, amd64...en savoir plus
- Type de serveur virtuel (type d'instance): t3.micro
- Pare-feu (groupe de sécurité): Nouveau groupe de sécurité
- Stockage (volumes): 1 volume(s) - 20 Gio

Buttons for 'Annuler' and 'Lancer l'instance' are visible at the bottom right of the configuration area.

Figure 6 b : Groupe de sécurité 2 et 3 (règle TCP aux ports 9870 et 8088)

3.4 Configuration du stockage

Un volume de 20 Go au format gp3 est configuré, ce qui est suffisant.

The screenshot displays the AWS Management Console interface for configuring the storage for an EC2 instance. The 'Configurer le stockage' section shows:

- 1x 20 Gio gp3 Volume racine, 3000 opérations d'E/S par seconde, Non chiffré.
- A button to 'Ajouter un volume'.
- A note: 'L'AMI sélectionnée contient des volumes de stockage d'instance, mais l'instance n'autorise aucun volume de stockage d'instance. Aucun des volumes de stockage d'instance provenant de l'AMI ne sera accessible depuis l'instance'.
- A warning: 'Cliquez sur Actualiser pour afficher les informations de sauvegarde. Les balises que vous attribuez déterminent si l'instance sera sauvegardée conformément aux stratégies de Data Lifecycle Manager.'
- Systeme de fichiers: Fichiers S3 - nouveau, EFS, FSx, and Aucun(e) (selected).

The summary (Récapitulatif) on the right indicates:

- Nombre d'instances: 1
- Image logicielle (AMI): Canonical, Ubuntu, 26.04, amd64...en savoir plus
- Type de serveur virtuel (type d'instance): t3.micro
- Pare-feu (groupe de sécurité): Nouveau groupe de sécurité
- Stockage (volumes): 1 volume(s) - 20 Gio

Buttons for 'Annuler' and 'Lancer l'instance' are visible at the bottom right of the configuration area.

Figure 7 : Configuration stockage à 20 Go SSD gp3

3.5 Lancement et démarrage de l'instance

Après validation de tous les paramètres, l'instance est lancée. Elle apparaît dans la liste EC2 avec l'état "En cours d'exécution" (running). Son identifiant unique est i-079945da204f0515e, déployée en zone de disponibilité af-south-1b.

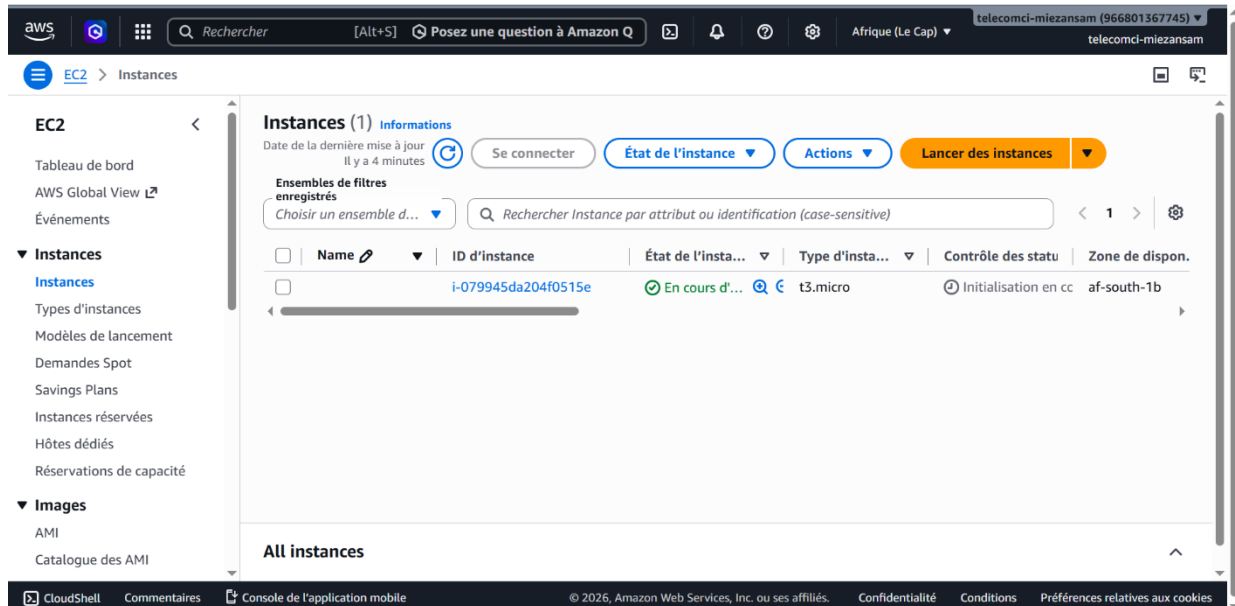


Figure 8 : Informations sur l'instance créée

4. Connexion SSH à l'instance

4.1 Configuration de PuTTY

PuTTY est configuré avec l'adresse IP publique de l'instance (13.246.27.100), le port 22, la clé privée hadoop-key.ppk et la session est sauvegardée sous le nom "hadoop" pour faciliter les reconnexions futures.

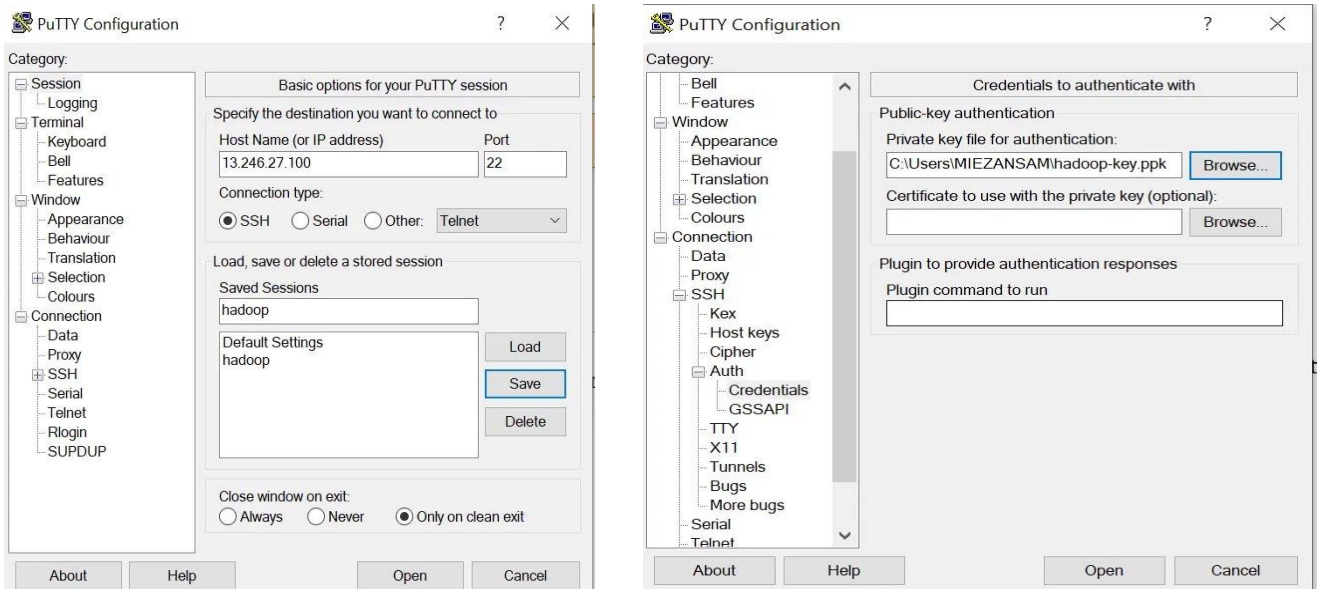


Figure 9 : Configuration PuTTY session "hadoop" avec IP publique et clé privée .ppk

4.2 Connexion établie

La connexion SSH est établie avec succès. Le terminal affiche le bandeau de bienvenue Ubuntu 26.04 LTS, confirmant l'authentification via la clé hadoop-key. L'IP privée interne de l'instance est 172.31.47.110.

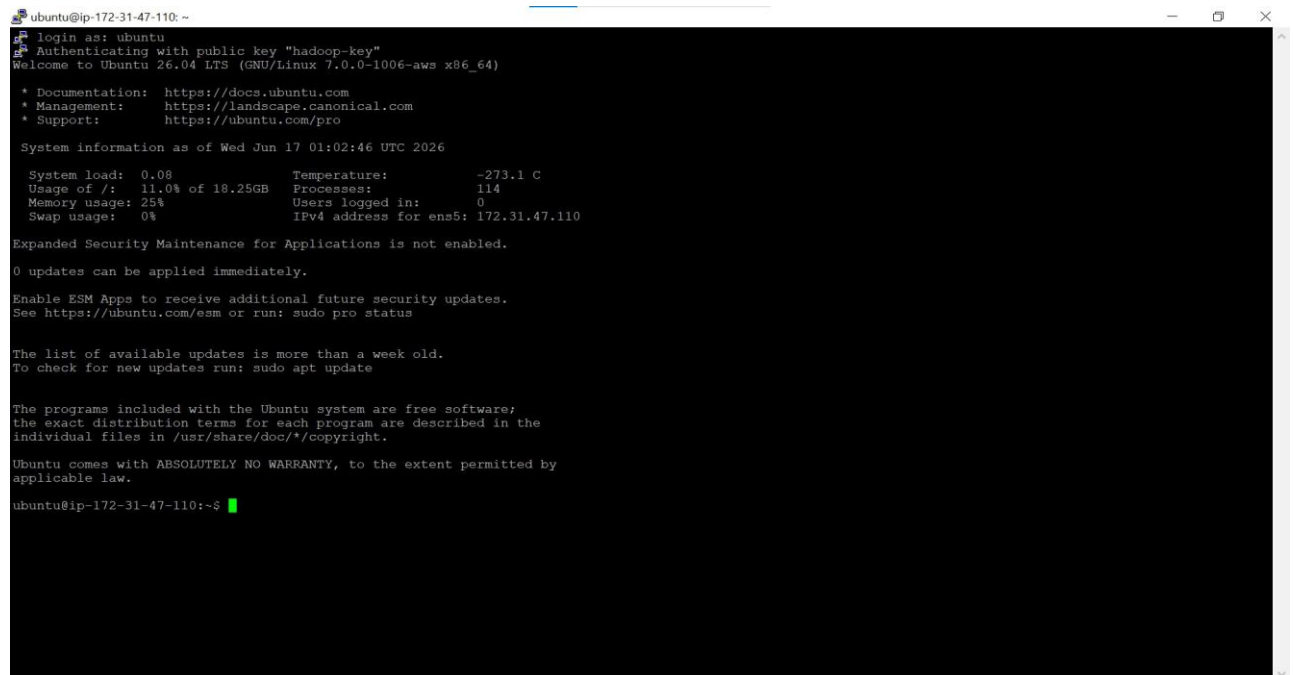


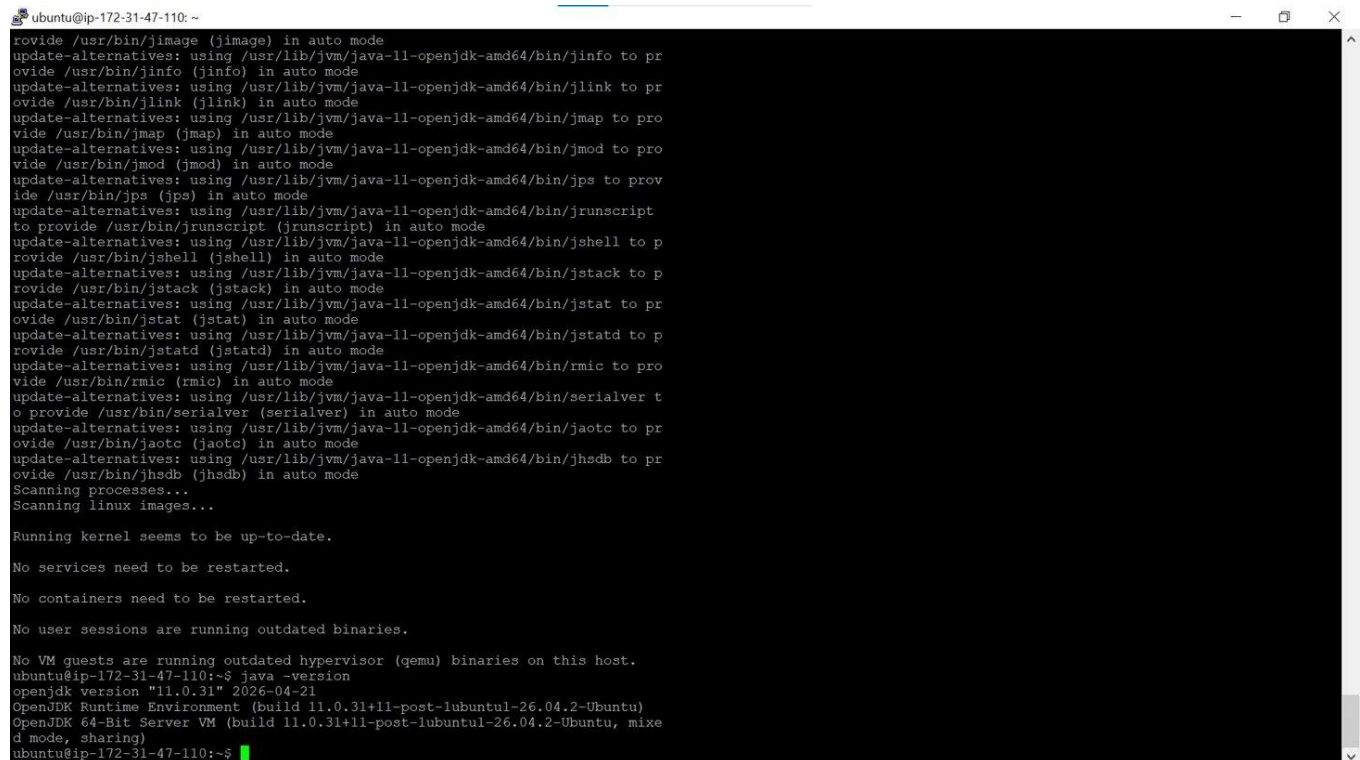
Figure 10 : Connexion SSH réussie bandeau Ubuntu 26.04 LTS, login "ubuntu"

5. Installation de Java 11

Hadoop 3.4.x requiert Java 8 ou Java 11. On installe OpenJDK 11 (version headless, sans interface graphique) via le gestionnaire de paquets apt d'Ubuntu.

Les commandes exécutées sont les suivantes :

```
sudo apt update
sudo apt install openjdk-11-jdk-headless -y
java -version
```



```
ubuntu@ip-172-31-47-110: ~
rovide /usr/bin/jimage (jimage) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jinfo to pr
ovide /usr/bin/jinfo (jinfo) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jlink to pr
ovide /usr/bin/jlink (jlink) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jmap to pr
ovide /usr/bin/jmap (jmap) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jmod to pr
ovide /usr/bin/jmod (jmod) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jps to pr
ovide /usr/bin/jps (jps) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jrunscript
to provide /usr/bin/jrunscript (jrunscript) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jshell to p
rovide /usr/bin/jshell (jshell) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jstack to p
rovide /usr/bin/jstack (jstack) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jstat to pr
ovide /usr/bin/jstat (jstat) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jstatd to p
rovide /usr/bin/jstatd (jstatd) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/rmic to pr
ovide /usr/bin/rmic (rmic) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/serialver t
o provide /usr/bin/serialver (serialver) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jaotc to pr
ovide /usr/bin/jaotc (jaotc) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jhsdb to pr
ovide /usr/bin/jhsdb (jhsdb) in auto mode
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-47-110:~$ java -version
openjdk version "11.0.31" 2026-04-21
OpenJDK Runtime Environment (build 11.0.31+11-post-lubuntu1-26.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 11.0.31+11-post-lubuntu1-26.04.2-Ubuntu, mixe
d mode, sharing)
ubuntu@ip-172-31-47-110:~$
```

Figure 11 : Installation Java réussie OpenJDK 11.0.31 confirmé par la commande "java -version"

6. Installation de Hadoop et des variables d'environnement

6.1 Téléchargement et extraction de Hadoop 3.4.3

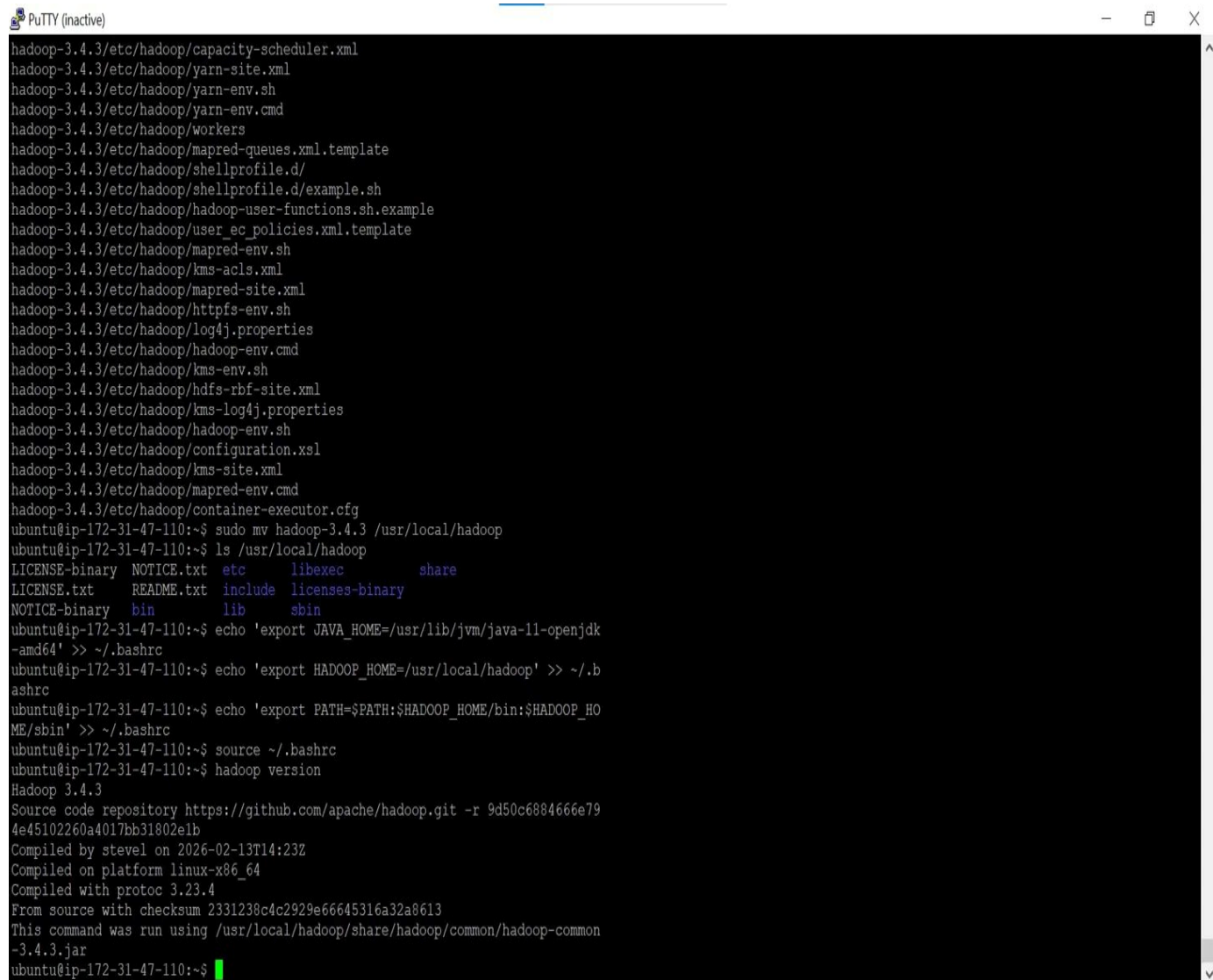
L'archive Hadoop 3.4.3 est téléchargée depuis les serveurs Apache officiels, extraite, puis déplacée vers le répertoire d'installation `/usr/local/hadoop`.

6.2 Configuration des variables d'environnement

Les variables d'environnement nécessaires sont ajoutées au fichier `~/.bashrc` pour être disponibles à chaque connexion SSH :

- `JAVA_HOME` : chemin vers le JDK Java 11
- `HADOOP_HOME` : chemin vers l'installation Hadoop
- `PATH` : ajout des répertoires `bin/` et `sbin/` de Hadoop

La commande `hadoop version` confirme que Hadoop 3.4.3 est correctement installé et accessible.



```
hadoop-3.4.3/etc/hadoop/capacity-scheduler.xml
hadoop-3.4.3/etc/hadoop/yarn-site.xml
hadoop-3.4.3/etc/hadoop/yarn-env.sh
hadoop-3.4.3/etc/hadoop/yarn-env.cmd
hadoop-3.4.3/etc/hadoop/workers
hadoop-3.4.3/etc/hadoop/mapred-queues.xml.template
hadoop-3.4.3/etc/hadoop/shellprofile.d/
hadoop-3.4.3/etc/hadoop/shellprofile.d/example.sh
hadoop-3.4.3/etc/hadoop/hadoop-user-functions.sh.example
hadoop-3.4.3/etc/hadoop/user_ec_policies.xml.template
hadoop-3.4.3/etc/hadoop/mapred-env.sh
hadoop-3.4.3/etc/hadoop/kms-acls.xml
hadoop-3.4.3/etc/hadoop/mapred-site.xml
hadoop-3.4.3/etc/hadoop/httpfs-env.sh
hadoop-3.4.3/etc/hadoop/log4j.properties
hadoop-3.4.3/etc/hadoop/hadoop-env.cmd
hadoop-3.4.3/etc/hadoop/kms-env.sh
hadoop-3.4.3/etc/hadoop/hdfs-rbf-site.xml
hadoop-3.4.3/etc/hadoop/kms-log4j.properties
hadoop-3.4.3/etc/hadoop/hadoop-env.sh
hadoop-3.4.3/etc/hadoop/configuration.xsl
hadoop-3.4.3/etc/hadoop/kms-site.xml
hadoop-3.4.3/etc/hadoop/mapred-env.cmd
hadoop-3.4.3/etc/hadoop/container-executor.cfg
ubuntu@ip-172-31-47-110:~$ sudo mv hadoop-3.4.3 /usr/local/hadoop
ubuntu@ip-172-31-47-110:~$ ls /usr/local/hadoop
LICENSE-binary  NOTICE.txt  etc          libexec      share
LICENSE.txt     README.txt  include      licenses-binary
NOTICE-binary   bin         lib          sbin
ubuntu@ip-172-31-47-110:~$ echo 'export JAVA_HOME=/usr/lib/jvm/java-11-openjdk
-amd64' >> ~/.bashrc
ubuntu@ip-172-31-47-110:~$ echo 'export HADOOP_HOME=/usr/local/hadoop' >> ~/.b
ashrc
ubuntu@ip-172-31-47-110:~$ echo 'export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HO
ME/sbin' >> ~/.bashrc
ubuntu@ip-172-31-47-110:~$ source ~/.bashrc
ubuntu@ip-172-31-47-110:~$ hadoop version
Hadoop 3.4.3
Source code repository https://github.com/apache/hadoop.git -r 9d50c6884666e79
4e45102260a4017bb31802e1b
Compiled by stevel on 2026-02-13T14:23Z
Compiled on platform linux-x86_64
Compiled with protoc 3.23.4
From source with checksum 2331238c4c2929e66645316a32a8613
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common
-3.4.3.jar
ubuntu@ip-172-31-47-110:~$
```

Figure 12 : Hadoop 3.4.3 installé variables d'environnement configurées, "hadoop version" validé

7. Configuration des Fichiers Hadoop

Hadoop en mode pseudo-distribué nécessite la configuration de quatre fichiers XML et du fichier d'environnement `hadoop-env.sh`. Ces fichiers définissent l'adresse du système de fichiers HDFS, le facteur de réplication, les répertoires de stockage des nœuds, et le framework de gestion des ressources.

7.1 Fichiers de configuration créés

Fichier	Paramètre principal configuré
core-site.xml	fs.defaultFS = hdfs://localhost:9000
hdfs-site.xml	dfs.replication=1, chemins namenode et datanode
mapred-site.xml	mapreduce.framework.name = yarn
yarn-site.xml	yarn.nodemanager.aux-services = mapreduce_shuffle
hadoop-env.sh	JAVA_HOME = /usr/lib/jvm/java-11-openjdk-amd64

```
ubuntu@ip-172-31-47-110: ~  
Summary:  
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 14  
  Download size: 41.0 kB  
  Space needed: 159 kB / 15.1 GB available  
  
Get:1 http://af-south-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 at amd64 3.2.5-2ubuntu2 [41.0 kB]  
Fetched 41.0 kB in 0s (250 kB/s)  
Selecting previously unselected package at.  
(Reading database ... 87239 files and directories currently installed.)  
Preparing to unpack .../at 3.2.5-2ubuntu2_amd64v3.deb ...  
Unpacking at (3.2.5-2ubuntu2) ...  
Setting up at (3.2.5-2ubuntu2) ...  
Created symlink '/etc/systemd/system/multi-user.target.wants/atd.service' -> '/usr/lib/systemd/system/atd.service'.  
Processing triggers for man-db (2.13.1-1build1) ...  
Scanning processes...  
Scanning linux images...  
  
Running kernel seems to be up-to-date.  
  
No services need to be restarted.  
  
No containers need to be restarted.  
  
No user sessions are running outdated binaries.  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
ubuntu@ip-172-31-47-110:~$ cat > /usr/local/hadoop/etc/hadoop/yarn-site.xml << 'EOF'  
<configuration>  
  <property>  
    <name>yarn.nodemanager.aux-services</name>  
    <value>mapreduce_shuffle</value>  
  </property>  
</configuration>  
EOF  
ubuntu@ip-172-31-47-110:~$ mkdir -p /usr/local/hadoop/data/namenode  
ubuntu@ip-172-31-47-110:~$ mkdir -p /usr/local/hadoop/data/datanode  
ubuntu@ip-172-31-47-110:~$ echo "Configuration terminée !"  
Configuration terminée !  
ubuntu@ip-172-31-47-110:~$ cat /usr/local/hadoop/etc/hadoop/core-site.xml  
<configuration>  
  <property>  
    <name>fs.defaultFS</name>  
    <value>hdfs://localhost:9000</value>  
  </property>  
</configuration>  
ubuntu@ip-172-31-47-110:~$
```

Figure 13 : Création des fichiers XML Hadoop et message "Configuration terminée !"


```
ubuntu@ip-172-31-47-110: ~  
</configuration>  
EOF  
ubuntu@ip-172-31-47-110:~$ mkdir -p /usr/local/hadoop/data/namenode  
ubuntu@ip-172-31-47-110:~$ mkdir -p /usr/local/hadoop/data/datanode  
ubuntu@ip-172-31-47-110:~$ echo "Configuration terminée !"  
Configuration terminée !  
ubuntu@ip-172-31-47-110:~$ cat /usr/local/hadoop/etc/hadoop/core-site.xml  
<configuration>  
  <property>  
    <name>fs.defaultFS</name>  
    <value>hdfs://localhost:9000</value>  
  </property>  
</configuration>  
ubuntu@ip-172-31-47-110:~$ cat /usr/local/hadoop/etc/hadoop/hdfs-site.xml  
cat /usr/local/hadoop/etc/hadoop/mapred-site.xml  
cat /usr/local/hadoop/etc/hadoop/yarn-site.xml  
tail -3 /usr/local/hadoop/etc/hadoop/hadoop-env.sh  
<configuration>  
  <property>  
    <name>dfs.replication</name>  
    <value>1</value>  
  </property>  
  <property>  
    <name>dfs.namenode.name.dir</name>  
    <value>/usr/local/hadoop/data/namenode</value>  
  </property>  
  <property>  
    <name>dfs.datanode.data.dir</name>  
    <value>/usr/local/hadoop/data/datanode</value>  
  </property>  
</configuration>  
<configuration>  
  <property>  
    <name>mapreduce.framework.name</name>  
    <value>yarn</value>  
  </property>  
</configuration>  
<configuration>  
  <property>  
    <name>yarn.nodemanager.aux-services</name>  
    <value>mapreduce_shuffle</value>  
  </property>  
</configuration>  
# server jvm.  
# export HADOOP_REGISTRYDNS_SECURE_EXTRA_OPTS="-jvm server"  
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64  
ubuntu@ip-172-31-47-110:~$
```

Figure 14 : Vérification du contenu des fichiers XML : core-site, hdfs-site, mapred-site, yarn-site

8. Configuration SSH local et formatage NameNode

8.1 Configuration SSH localhost

Hadoop doit pouvoir se connecter en SSH à localhost pour démarrer ses daemons, même en mode single-node. Une paire de clés RSA est générée localement, et la clé publique est ajoutée aux `authorized_keys`.

La connexion SSH vers localhost est testée et validée (empreinte confirmée avec "yes"). Une session locale s'ouvre avec succès, puis on sort avec la commande `exit`.

```
ubuntu@ip-172-31-47-110: ~
Your identification has been saved in /home/ubuntu/.ssh/id_rsa
Your public key has been saved in /home/ubuntu/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:JN04G9hEfsIfFGpqtWotTnri67iHEunJCIMt+XPUfYI ubuntu@ip-172-31-47-110
The key's randomart image is:
+---[RSA 3072]-----+
|  .o o. |
|    B =  |
|  o # +  |
|   o B @ . |
| .+ . + E + . |
|o. o o o o |
| =...o * . |
|o=..oB.. |
| o==+. |
+---[SHA256]-----+
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is: SHA256:0bkVZSrEl0Wn6QzjNwCXUa5EFQJoGfbVfqGNAfle2+Q
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Jun 17 02:03:57 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   23.0% of 18.25GB Processes:    124
Memory usage: 32%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.47.110

Expanded Security Maintenance for Applications is not enabled.

14 updates can be applied immediately.
14 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed Jun 17 02:03:58 2026 from 102.209.221.134
ubuntu@ip-172-31-47-110:~$
```

Figure 15 : Génération clé RSA locale, connexion SSH localhost validée

8.2 Formatage du NameNode

Le NameNode HDFS doit être formaté une seule fois, lors de la première installation. Cette opération initialise les métadonnées du système de fichiers distribué.

La commande `hdfs namenode -format` génère les journaux d'initialisation et confirme le succès avec le message : "Storage directory /usr/local/hadoop/data/namenode has been successfully formatted."

Rapport d'Installation Hadoop sur AWS EC2

```
ubuntu@ip-172-31-47-110: ~  
2026-06-17 02:06:21,146 INFO snapshot.SnapshotManager: Loaded config captureOpenFiles: false, skipCaptureAccessTimeOnlyChange: false, snapshotDiffAllowSnapRo  
otDescendant: true, maxSnapshotFSLimit: 65536, maxSnapshotLimit: 65536  
2026-06-17 02:06:21,146 INFO snapshot.SnapshotManager: dfs.namenode.snapshot.deletion.ordered = false  
2026-06-17 02:06:21,149 INFO snapshot.SnapshotManager: SkipList is disabled  
2026-06-17 02:06:21,156 INFO util.GSet: Computing capacity for map cachedBlocks  
2026-06-17 02:06:21,156 INFO util.GSet: VM type = 64-bit  
2026-06-17 02:06:21,157 INFO util.GSet: 0.25% max memory 220.4 MB = 564.3 KB  
2026-06-17 02:06:21,157 INFO util.GSet: capacity = 2^16 = 65536 entries  
2026-06-17 02:06:21,173 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.window.num.buckets = 10  
2026-06-17 02:06:21,174 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.num.users = 10  
2026-06-17 02:06:21,174 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.windows.minutes = 1,5,25  
2026-06-17 02:06:21,179 INFO namenode.FSNamesystem: Retry cache on namenode is enabled  
2026-06-17 02:06:21,180 INFO namenode.FSNamesystem: Retry cache will use 0.03 of total heap and retry cache entry expiry time is 600000 millis  
2026-06-17 02:06:21,186 INFO util.GSet: Computing capacity for map NameNodeRetryCache  
2026-06-17 02:06:21,186 INFO util.GSet: VM type = 64-bit  
2026-06-17 02:06:21,186 INFO util.GSet: 0.029999999329447746% max memory 220.4 MB = 67.7 KB  
2026-06-17 02:06:21,186 INFO util.GSet: capacity = 2^13 = 8192 entries  
2026-06-17 02:06:21,230 INFO namenode.FSImage: Allocated new BlockPoolId: BP-1630263747-172.31.47.110-1781661981221  
2026-06-17 02:06:21,263 INFO common.Storage: Storage directory /usr/local/hadoop/data/namenode has been successfully formatted.  
2026-06-17 02:06:21,334 INFO namenode.FSImageFormatProtobuf: Saving image file /usr/local/hadoop/data/namenode/current/fsimage.ckpt_000000000000000000 using  
no compression  
2026-06-17 02:06:21,528 INFO namenode.FSImageFormatProtobuf: Image file /usr/local/hadoop/data/namenode/current/fsimage.ckpt_000000000000000000 of size 398  
bytes saved in 0 seconds .  
2026-06-17 02:06:21,548 INFO namenode.NNStorageRetentionManager: Going to retain 1 images with txid >= 0  
2026-06-17 02:06:21,557 INFO blockmanagement.DatanodeManager: Slow peers collection thread shutdown  
2026-06-17 02:06:21,563 INFO namenode.FSNamesystem: Stopping services started for active state  
2026-06-17 02:06:21,563 INFO namenode.FSNamesystem: Stopping services started for standby state  
2026-06-17 02:06:21,570 INFO namenode.FSImage: FSImageSaver clean checkpoint: txid=0 when meet shutdown.  
2026-06-17 02:06:21,571 INFO namenode.NameNode: SHUTDOWN MSG:  
/*****  
SHUTDOWN MSG: Shutting down NameNode at ip-172-31-47-110/172.31.47.110  
*****/  
ubuntu@ip-172-31-47-110:~$ start-dfs.sh  
start-yarn.sh  
jps  
Starting namenodes on [localhost]  
Starting datanodes  
Starting secondary namenodes [ip-172-31-47-110]  
ip-172-31-47-110: Warning: Permanently added 'ip-172-31-47-110' (ED25519) to the list of known hosts.  
Starting resourcemanager  
Starting nodemanagers  
4481 NodeManager  
3826 NameNode  
4359 ResourceManager  
4584 Jps  
4110 SecondaryNameNode  
ubuntu@ip-172-31-47-110:~$
```

Figure 16 : Formatage NameNode réussi + lancement DFS et YARN

Remarque : Cette commande ne doit être exécutée qu'une seule fois. Reformater le NameNode efface irrémédiablement toutes les données HDFS existantes.

9. Premier démarrage et détection du problème OOM

9.1 Résultat du premier démarrage

Après le formatage, les daemons HDFS et YARN sont lancés avec start-dfs.sh et start-yarn.sh. La commande jps (Java Process Status) est utilisée pour lister les processus Java actifs.

Résultat observé : le DataNode et le ResourceManager sont absents de la liste jps, malgré un démarrage sans message d'erreur apparent. Seuls NameNode, SecondaryNameNode et NodeManager apparaissent.

```
ubuntu@ip-172-31-47-110: ~  
e  
2026-06-17 02:09:05,839 INFO org.apache.hadoop.yarn.nodelabels.store.AbstractFSNodeStore: Finished write mirror at:file:/tmp/hadoop-yarn-ubuntu/node-attribute/nodeattribute.mirror  
2026-06-17 02:09:05,845 INFO org.apache.hadoop.yarn.nodelabels.store.AbstractFSNodeStore: Finished create editlog file at:file:/tmp/hadoop-yarn-ubuntu/node-attribute/nodeattribute.editlog  
2026-06-17 02:09:05,870 INFO org.apache.hadoop.yarn.event.AsyncDispatcher: Registering class org.apache.hadoop.yarn.server.resourcemanager.nodelabels.NodeAttributesStoreEventType for class org.apache.hadoop.yarn.server.resourcemanager.nodelabels.NodeAttributesManagerImpl$ForwardingEventHandler  
2026-06-17 02:09:05,896 INFO org.apache.hadoop.yarn.server.resourcemanager.scheduler.placement.MultiNodeSortingManager: Starting NodeSortingService=MultiNodeSortingManager  
2026-06-17 02:09:06,099 INFO org.apache.hadoop.ipc.CallQueueManager: Using callQueue: class java.util.concurrent.LinkedBlockingQueue, queueCapacity: 5000, scheduler: class org.apache.hadoop.ipc.DefaultRpcScheduler, ipcBackoff: false, ipcFailOver: false.  
2026-06-17 02:09:06,112 INFO org.apache.hadoop.ipc.Server: Listener at 0.0.0.0:8031  
2026-06-17 02:09:06,125 INFO org.apache.hadoop.ipc.Server: Starting Socket Reader #1 for port 8031  
2026-06-17 02:09:06,244 INFO org.apache.hadoop.yarn.factory.impl.pb.RpcServerFactoryPBImpl: Adding protocol org.apache.hadoop.yarn.server.api.ResourceTrackerPB to the server  
2026-06-17 02:09:06,307 INFO org.apache.hadoop.ipc.Server: IPC Server listener on 8031: starting  
2026-06-17 02:09:06,307 INFO org.apache.hadoop.ipc.Server: IPC Server Responder: starting  
2026-06-17 02:09:11,328 INFO org.apache.hadoop.util.JvmPauseMonitor: Starting JVM pause monitor  
2026-06-17 02:09:20,979 INFO org.apache.hadoop.ipc.CallQueueManager: Using callQueue: class java.util.concurrent.LinkedBlockingQueue, queueCapacity: 5000, scheduler: class org.apache.hadoop.ipc.DefaultRpcScheduler, ipcBackoff: false, ipcFailOver: false.  
2026-06-17 02:09:28,883 INFO org.apache.hadoop.ipc.Server: Listener at 0.0.0.0:8030  
2026-06-17 02:09:29,124 INFO org.apache.hadoop.ipc.Server: Starting Socket Reader #1 for port 8030  
ubuntu@ip-172-31-47-110:~$ jps  
4833 Jps  
4481 NodeManager  
3826 NameNode  
4110 SecondaryNameNode  
ubuntu@ip-172-31-47-110:~$ jps  
4481 NodeManager  
3826 NameNode  
4854 Jps  
4110 SecondaryNameNode  
ubuntu@ip-172-31-47-110:~$ free -h  
sudo dmesg | grep -i "out of memory"  
sudo dmesg | grep -i "killed process"  
              total        used        free      shared  buff/cache   available  
Mem:           908Mi       768Mi        97Mi        2.9Mi       151Mi       140Mi  
Swap:              0B           0B           0B  
[ 5501.587356] Out of memory: Killed process 3940 (java) total-vm:2002628kB, anon-rss:144340kB, file-rss:2700kB, shmem-rss:32kB, UID:1000 pgtables:596kB oom_score_adj:0  
[ 5571.157474] Out of memory: Killed process 4359 (java) total-vm:2069860kB, anon-rss:182704kB, file-rss:2752kB, shmem-rss:32kB, UID:1000 pgtables:788kB oom_score_adj:0  
[ 5501.587356] Out of memory: Killed process 3940 (java) total-vm:2002628kB, anon-rss:144340kB, file-rss:2700kB, shmem-rss:32kB, UID:1000 pgtables:596kB oom_score_adj:0  
[ 5571.157474] Out of memory: Killed process 4359 (java) total-vm:2069860kB, anon-rss:182704kB, file-rss:2752kB, shmem-rss:32kB, UID:1000 pgtables:788kB oom_score_adj:0  
ubuntu@ip-172-31-47-110:~$
```

Figure 17 : Premier jps avec DataNode et ResourceManager absents (problème détecté)

9.2 Diagnostic : OOM Killer Linux

Le diagnostic est effectué avec les commandes free -h (état de la mémoire) et sudo dmesg | grep "out of memory" (journaux noyau). Les résultats révèlent :

- RAM totale : 908 Mi, RAM utilisée : 768 Mi, Swap : 0 Mo (aucun)
- Journaux dmesg : "Out of memory: Killed process 3940 (java)" → DataNode tué
- Journaux dmesg : "Out of memory: Killed process 4359 (java)" → ResourceManager tué

Le mécanisme OOM Killer (Out of Memory Killer) du noyau Linux a terminé de force les processus Java qui consommaient trop de mémoire, faute de RAM disponible et en l'absence de swap configuré.

```
ubuntu@ip-172-31-47-110: ~
score adj:0
[ 5571.157474] Out of memory: Killed process 4359 (java) total-vm:2069860kB, anon-rss:182704kB, file-rss:2752kB, shmem-rss:32kB, UID:1000 pgtables:788kB oom_
score adj:0
ubuntu@ip-172-31-47-110:~$ stop-yarn.sh
stop-dfs.sh
Stopping nodemanagers
localhost: WARNING: nodemanager did not stop gracefully after 5 seconds: Trying to kill with kill -9
Stopping resourcemanager
Stopping namenodes on [localhost]
Stopping datanodes
Stopping secondary namenodes [ip-172-31-47-110]
ubuntu@ip-172-31-47-110:~$ sudo fallocation -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile swap swap defaults 0 0' | sudo tee -a /etc/fstab
Setting up swappiness version 1, size = 2 GiB (2147479552 bytes)
no label, UUID=a7612896-9ddc-42b8-a0d2-570bed36b46b
/swapfile swap swap defaults 0 0
ubuntu@ip-172-31-47-110:~$ echo 'export HADOOP_HEAPSIZE_MAX=256' >> /usr/local/hadoop/etc/hadoop/hadoop-env.sh
ubuntu@ip-172-31-47-110:~$ echo 'export YARN_RESOURCEMANAGER_HEAPSIZE=256' >> /usr/local/hadoop/etc/hadoop/yarn-env.sh
echo 'export YARN_NODEMANAGER_HEAPSIZE=256' >> /usr/local/hadoop/etc/hadoop/yarn-env.sh
ubuntu@ip-172-31-47-110:~$ cat > /usr/local/hadoop/etc/hadoop/yarn-site.xml << 'EOF'
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.resource.memory-mb</name>
    <value>768</value>
  </property>
  <property>
    <name>yarn.scheduler.maximum-allocation-mb</name>
    <value>768</value>
  </property>
  <property>
    <name>yarn.scheduler.minimum-allocation-mb</name>
    <value>128</value>
  </property>
</configuration>
EOF
ubuntu@ip-172-31-47-110:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           908Mi       329Mi       496Mi       2.8Mi       191Mi       578Mi
Swap:          2.0Gi          0B        2.0Gi
ubuntu@ip-172-31-47-110:~$
```

Figure 18 : Diagnostic OOM dmesg confirme les processus Java tués par manque de RAM

10. Correctif mémoire (Partie 9b du document)

Le correctif mémoire est appliqué en 5 étapes avant de relancer les daemons Hadoop.

10.1 Arrêt des daemons et ajout de swap

Les daemons sont d'abord arrêtés proprement (stop-yarn.sh + stop-dfs.sh), puis un fichier de swap de 2 Go est créé, formaté et activé de manière permanente (ajout dans /etc/fstab).

10.2 Réduction des heaps Java

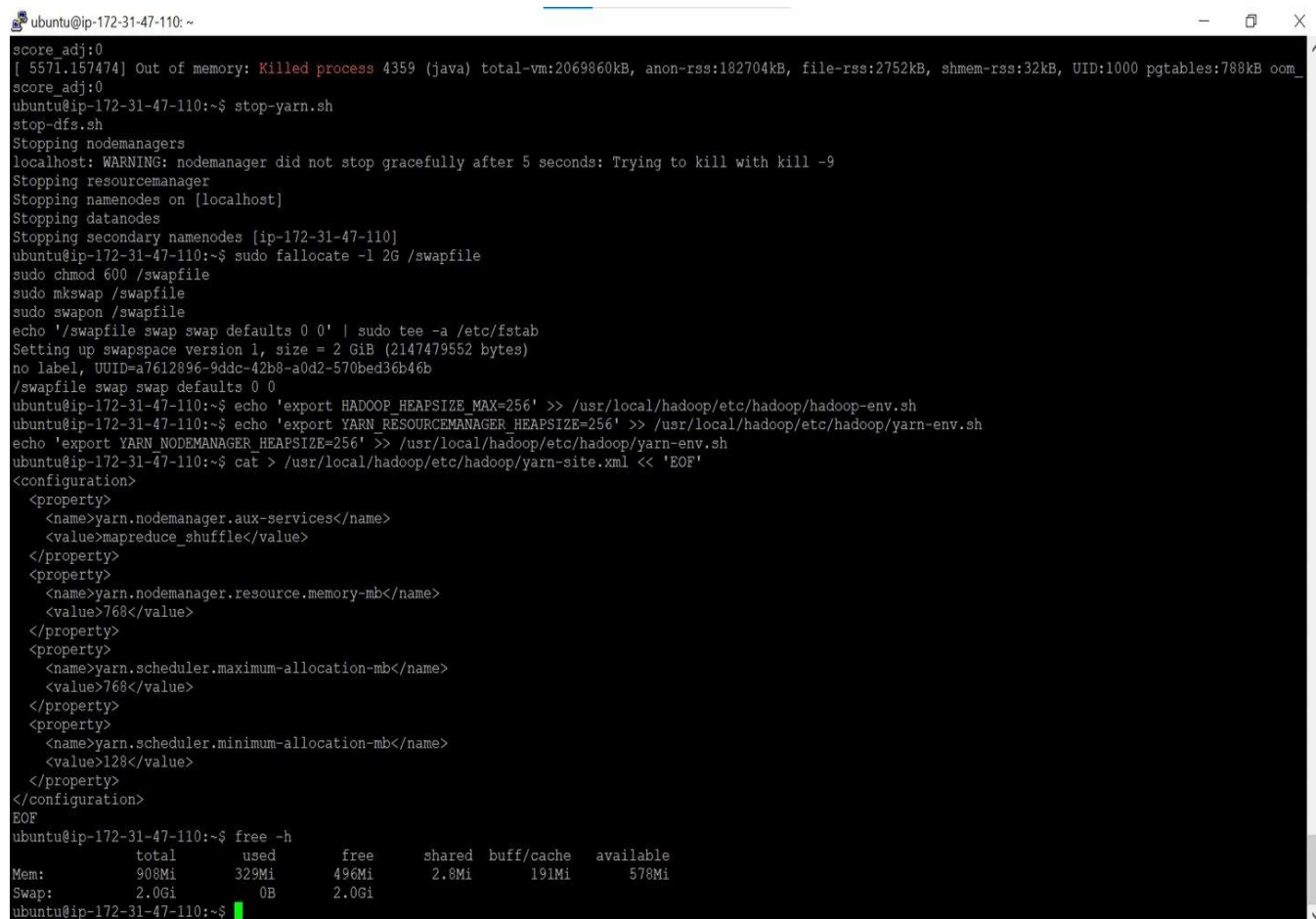
Les variables d'environnement suivantes sont ajoutées pour limiter la mémoire allouée par la JVM à chaque daemon :

- hadoop-env.sh : HADOOP_HEAPSIZE_MAX=256 (limite tous les daemons HDFS à 256 Mo)
- yarn-env.sh : YARN_RESOURCEMANAGER_HEAPSIZE=256 et YARN_NODEMANAGER_HEAPSIZE=256

10.3 Limitation mémoire des conteneurs YARN

Le fichier yarn-site.xml est réécrit avec des paramètres de mémoire adaptés à l'instance 1 Go :

- yarn.nodemanager.resource.memory-mb = 768 Mo
- yarn.scheduler.maximum-allocation-mb = 768 Mo
- yarn.scheduler.minimum-allocation-mb = 128 Mo



```
ubuntu@ip-172-31-47-110: ~
score_adj:0
[ 5571.157474] Out of memory: Killed process 4359 (java) total-vm:2069860kB, anon-rss:182704kB, file-rss:2752kB, shmem-rss:32kB, UID:1000 pgtables:788kB oom
score_adj:0
ubuntu@ip-172-31-47-110:~$ stop-yarn.sh
stop-dfs.sh
Stopping nodemanagers
localhost: WARNING: nodemanager did not stop gracefully after 5 seconds: Trying to kill with kill -9
Stopping resourcemanager
Stopping namenodes on [localhost]
Stopping datanodes
Stopping secondary namenodes [ip-172-31-47-110]
ubuntu@ip-172-31-47-110:~$ sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile swap swap defaults 0 0' | sudo tee -a /etc/fstab
Setting up swap space version 1, size = 2 GiB (2147479552 bytes)
no label, UUID=a7612896-9ddc-42b8-a0d2-570bed36b46b
/swapfile swap swap defaults 0 0
ubuntu@ip-172-31-47-110:~$ echo 'export HADOOP_HEAPSIZE_MAX=256' >> /usr/local/hadoop/etc/hadoop/hadoop-env.sh
ubuntu@ip-172-31-47-110:~$ echo 'export YARN_RESOURCEMANAGER_HEAPSIZE=256' >> /usr/local/hadoop/etc/hadoop/yarn-env.sh
echo 'export YARN_NODEMANAGER_HEAPSIZE=256' >> /usr/local/hadoop/etc/hadoop/yarn-env.sh
ubuntu@ip-172-31-47-110:~$ cat > /usr/local/hadoop/etc/hadoop/yarn-site.xml << 'EOF'
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.resource.memory-mb</name>
    <value>768</value>
  </property>
  <property>
    <name>yarn.scheduler.maximum-allocation-mb</name>
    <value>768</value>
  </property>
  <property>
    <name>yarn.scheduler.minimum-allocation-mb</name>
    <value>128</value>
  </property>
</configuration>
EOF
ubuntu@ip-172-31-47-110:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           908Mi       329Mi       496Mi       2.8Mi       191Mi       578Mi
Swap:          2.0Gi          0B        2.0Gi
ubuntu@ip-172-31-47-110:~$
```

Figure 19 : Application du correctif swap 2 Go créé, heaps réduits, yarn-site.xml reconfiguré

Après application du correctif, la commande `free -h` confirme :

- RAM : 908 Mi totale, 329 Mi utilisée, 496 Mi libre
- Swap : 2.0 Gi totale, 0 utilisé, 2.0 Gi disponible. Le filet de sécurité est en place

Remarque : Le correctif mémoire est correctement appliqué. La mémoire swap de 2 Go est disponible et les heaps Java sont limités à 256 Mo par daemon.

11. Démarrage final et validation du Cluster

11.1 Vérification des services Hadoop via les interfaces Web

Après le redémarrage du cluster, l'interface Web du NameNode accessible sur le port 9870 confirme que le système de fichiers HDFS est opérationnel. L'état "active" indique que le NameNode fonctionne correctement et assure la gestion des métadonnées du système distribué.

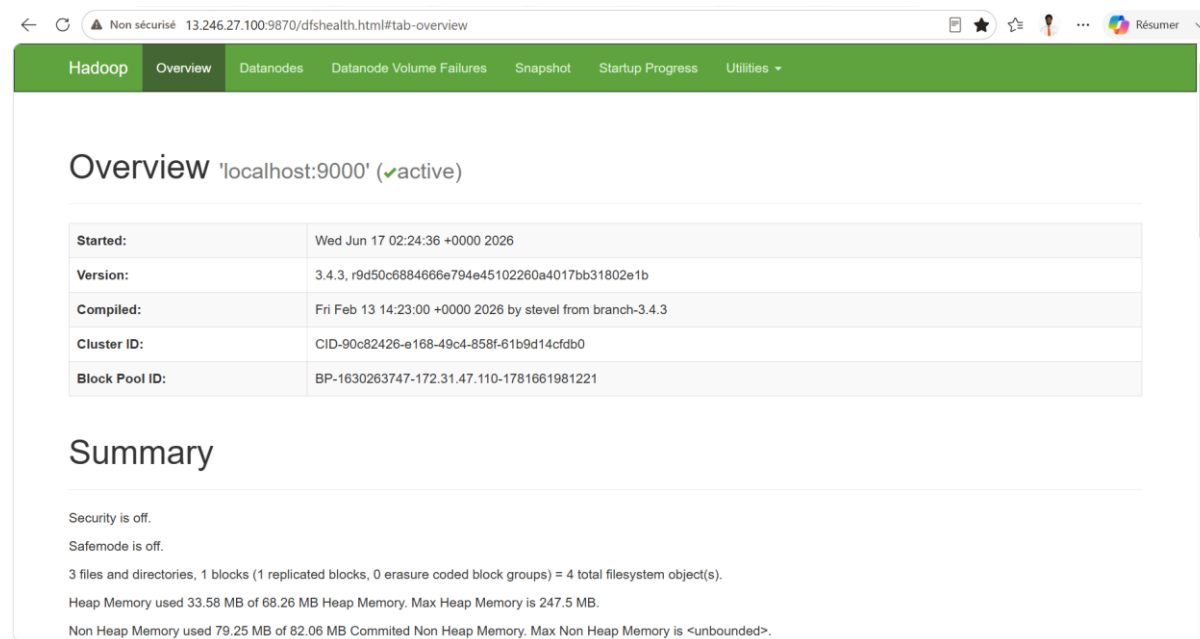


Figure 20 : Interface Web du NameNode HDFS

L'interface Web du ResourceManager accessible sur le port 8088 valide le fonctionnement de YARN. Le cluster possède un nœud actif et les ressources mémoire configurées correspondent aux limitations appliquées précédemment pour l'instance EC2 t3.micro.

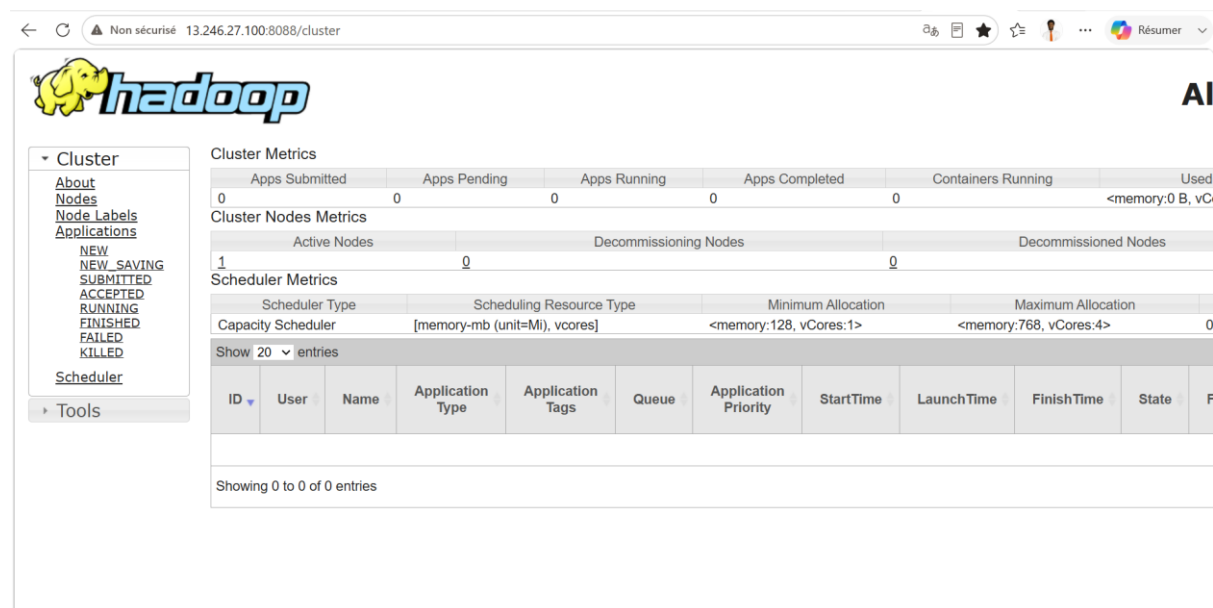


Figure 21 : Interface Web du ResourceManager YARN

11.2 Deuxième démarrage avec 5 daemons actifs

Avec les corrections mémoire en place, Hadoop est relancé. Après un délai de 15 secondes (sleep 15) pour laisser le temps à tous les daemons de s'initialiser, la commande jps affiche bien les 5 processus requis :

PID	Daemon	Rôle
5852	NameNode	Gestionnaire des métadonnées HDFS
5972	DataNode	Stockage effectif des blocs de données
6175	SecondaryNameNode	Sauvegarde des checkpoints NameNode
6389	ResourceManager	Gestion des ressources cluster (YARN)
6517	NodeManager	Gestion des ressources du nœud (YARN)

```

ubuntu@ip-172-31-47-110: ~
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.resource.memory-mb</name>
    <value>768</value>
  </property>
  <property>
    <name>yarn.scheduler.maximum-allocation-mb</name>
    <value>768</value>
  </property>
  <property>
    <name>yarn.scheduler.minimum-allocation-mb</name>
    <value>128</value>
  </property>
</configuration>
EOF
ubuntu@ip-172-31-47-110:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:            908Mi       329Mi       496Mi        2.8Mi       191Mi       578Mi
Swap:           2.0Gi          0B        2.0Gi
ubuntu@ip-172-31-47-110:~$ start-dfs.sh
start-yarn.sh
sleep 15
jps
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [ip-172-31-47-110]
Starting resourcemanager
Starting nodemanagers
6880 Jps
5972 DataNode
6517 NodeManager
6389 ResourceManager
5852 NameNode
6175 SecondaryNameNode
ubuntu@ip-172-31-47-110:~$ hdfs dfs -mkdir /test
hdfs dfs -ls /
echo "Hello Hadoop" > test.txt
hdfs dfs -put test.txt /test/
hdfs dfs -cat /test/test.txt
Found 1 items
drwxr-xr-x  - ubuntu supergroup      0 2026-06-17 02:27 /test
Hello Hadoop
ubuntu@ip-172-31-47-110:~$

```

Figure 22 : Deuxième démarrage jps affiche les 5 daemons Hadoop actifs + test HDFS réussi

11.3 Test pratique HDFS (écriture et lecture)

Un test fonctionnel complet est réalisé sur HDFS pour valider l'installation :

```
hdfs dfs -mkdir /test
hdfs dfs -ls /
echo "Hello Hadoop" > test.txt
hdfs dfs -put test.txt /test/
hdfs dfs -cat /test/test.txt
```

Résultat : "Hello Hadoop" s'affiche dans le terminal, validant que le fichier a bien été écrit puis lu depuis HDFS.

Le Test HDFS complet est réussi et on a une création de répertoire, upload et lecture d'un fichier. Le cluster Hadoop est donc pleinement opérationnel.

12. Conclusion

Ce rapport documente l'installation complète et réussie d'Apache Hadoop 3.4.3 en mode pseudo-distribué sur une instance AWS EC2 (Ubuntu 26.04 LTS, t3.micro). L'ensemble des étapes a été réalisé, allant de la création de l'instance virtuelle jusqu'à la validation des services HDFS et YARN, a été réalisé avec succès. Des captures d'écran attestant de la validation de chaque phase du processus sont inclus dans le rapport.

Cette installation a permis de comprendre le fonctionnement des principaux composants Hadoop, notamment le rôle du NameNode et du DataNode dans la gestion du stockage distribué, ainsi que celui du ResourceManager et du NodeManager dans l'orchestration des ressources.

Une difficulté majeure a été rencontrée lors du premier démarrage du cluster avec l'apparition du problème OOM Killer, causé par les ressources limitées de l'instance EC2 t3.micro disposant de seulement 1 Go de RAM. L'analyse des journaux système a permis d'identifier précisément l'origine du problème. La mise en place d'une mémoire swap de 2 Go, associée à la réduction des heaps JVM à 256 Mo et à l'ajustement des paramètres YARN, a permis d'obtenir un cluster stable avec les cinq daemons Hadoop actifs.